

课程笔记

Codex Skills 与 Tools 加载、Agent Runtime 全景

基于公开课程资料整理

2025

modern AI Agent Skills 与 Tools Agent Runtime Agent Loop

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 11 分钟

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 Modern AI Agents 的定义与特征	2
1.1 什么是 Modern AI Agent	2
1.2 Context Engineering 的核心地位	2
1.3 本章小结	2
2 Skills 与 MCP/Tools 的正交关系	2
2.1 两种不同的 Context 位置	2
2.2 Skills 的渐进式加载回顾	3
2.3 本章小结	3
3 Agent Runtime 与 Agent Loop	3
3.1 Agent Runtime: Agent 的运行时环境	3
3.2 Agent Loop: 消息驱动的执行循环	4
3.3 本章小结	4
4 Codex 内置 Tools 全景	4
4.1 Tools Schema 的获取方法	4
4.2 Codex 的五类内置 Tools	5
4.3 Multi-Agent (子代理) 工具	5
4.4 MCP 作为 Dynamic Tools	6
4.5 本章小结	6
5 总结与延伸	6
5.1 Context Engineering 的全景图	6
5.2 拓展阅读	7

1 Modern AI Agents 的定义与特征

1.1 什么是 Modern AI Agent

Codex、Claude Code、Gemini CLI、OpenAI Codex 等工具都属于 Modern AI Agent。与早期的 Agent 框架相比，它们具有三个显著特征：

Modern AI Agent 的三大特征

1. **更强的基础模型**：底层使用 GPT-4.5、Opus 4.6 等前沿模型，具备强大的推理和代码生成能力
2. **更长程的 Agentic 多轮能力**：用户发出一个 Query 后，Agent 可以进行漫长的多轮长程交互，自主调用工具、读写文件、执行命令，最终完成复杂任务
3. **精心设计的 Agent Runtime**：大量的提示词注入 (agents.md、src.md、memory.md)、丰富的内置 Tools、Skills 系统、Spawn Agent / Multi-Agent 设计、完善的调度编排系统

1.2 Context Engineering 的核心地位

所有 Modern AI Agent 的本质

所有 Modern AI Agent 本质上都在做同一件事——**Context Engineering**。它们维护一个消息列表 (Message List)，通过精心设计消息的内容、位置和加载时机，构建远端或本地大模型推理所需的完整上下文。

关键原则：不管是 Skills 还是 MCP 工具，如果它们不暴露在 Context (API 的 Message List) 中，模型就**不可能**感知到这些 Skills 和 Tools 的存在，也就无从调用。

1.3 本章小结

Modern AI Agent 的核心是 Context Engineering——维护和管理 Agent Loop 执行过程中的上下文，包括预注入指令、Skills 暴露、Tools 注册、对话历史管理等。

2 Skills 与 MCP/Tools 的正交关系

2.1 两种不同的 Context 位置

Skills 和 MCP/Tools 在技术架构上是**正交独立**的，它们占据 API 请求中完全不同的位置：

Skills vs. Tools 的位置差异

- **Skills**: 出现在 `message` 的 `content` 字段中, 具体是注入的第三条 User 消息, 与 `agents.md` 放在一起
- **MCP / 内置 Tools**: 出现在 API 的 `tools` 字段中, 以 Tools Schema 的形式暴露给模型

两者处于完全不同的位置, **不存在** Skills 出现后简化了 MCP/Tools 所占 Context 的情况 (除非显式设计)。

2.2 Skills 的渐进式加载回顾

以 Codex 为例, Skills 的加载流程:

1. 预注入的 User 消息中包含 Skills 的索引——每个 Skill 的名称和简短描述
2. 用户发出第一个 Query 后, 远端模型根据 Query 匹配最合适的 Skill
3. 如果决定使用某个 Skill, 模型通过 Function Call 执行一个 Shell 命令 (如 `cat`), 将该 Skill 对应的 `skills.md` 完整加载
4. 完整的 Skill 内容作为 Function Output 消息加入 Context

Skills 可以引用 Tools

Skills 的文档中可以提及具体的 Tools——说明什么时候应该使用什么工具。但这只是指引性的文本, 实际的 Tool 定义仍然需要通过 API 的 `tools` 字段以 Schema 形式暴露给模型。Skills 是”说明书”, Tools 是”工具箱”, 两者配合使用。

2.3 本章小结

Skills 和 MCP/Tools 在 API 层面占据不同位置——Skills 在消息内容中, Tools 在工具字段中。两者技术上正交独立, 但在实际使用中协同配合。

3 Agent Runtime 与 Agent Loop

3.1 Agent Runtime: Agent 的运行时环境

Agent Runtime 是 Agent 执行的”舞台”, 它提供 Agent 运行所需的一切环境支持:

- **Workspace**: 如 OpenAI Codex 的工作空间
- **启动文件注入**: 系统级的提示词 (System Prompt)
- **内置 Tools 和 Skills**: 预定义的工具集和技能文档
- **配置文件**: `agents.md`、`src.md`、`tools.md` 等 Markdown 配置文件
- **Memory 系统**: 记忆和持久化机制

3.2 Agent Loop: 消息驱动的执行循环

Agent Loop 描述了当一条消息真正进入系统后, 如何被处理并最终生成响应或动作:

Agent Loop 的标准流程

1. **Intake**: 接收用户消息 (可能需要排队)
2. **Prompt Injection**: 大量的提示词注入 (系统指令、Skills、环境信息等)
3. **Model Inference**: 调用远端或本地大模型进行推理
4. **Function Call** (可选): 如果模型决定调用工具, 执行对应的 Function
5. **Streaming Response**: 流式输出回复
6. **Persistence**: 持久化到本地 (保存对话历史)

这是以 OpenAI Codex 官方文档描述的 Agent Loop 为基础的标准流程。

Agent Runtime vs. Agent Loop

- **Agent Runtime**: 静态的运行时环境, 在 Agent 启动时准备好, 提供执行所需的一切资源
- **Agent Loop**: 动态的执行循环, 当消息进入后驱动 Agent 执行动作和生成回复

Runtime 是舞台, Loop 是舞台上演出的过程。

3.3 本章小结

Agent Runtime 提供运行时环境 (workspace、配置文件、内置工具), Agent Loop 是消息驱动的执行循环 (接收 → 注入 → 推理 → 调用 → 响应 → 持久化)。两者共同构成 Modern Agent 的完整架构。

4 Codex 内置 Tools 全景

4.1 Tools Schema 的获取方法

默认情况下, Codex 的 API 层面 Tools Schema 不会持久化到本地。如果需要查看完整的请求体 (包括 Tools 定义), 需要设置特定的环境变量来启用日志记录:

日志落盘方法

设置环境变量后, Codex 会将完整的 API 请求体保存到本地的 `logs-1.sqlite` 数据库中。通过查询数据库记录, 可以获取到包含 `instructions` (System Prompt)、`input` (Message List) 和 `tools` (Tools Schema) 的完整请求体。

请求体的结构 (Response API 格式):

- `instructions`: 独立的顶层指令, 相当于 System Prompt

- **input**: 完整的消息列表, 可包含 Developer、User、Assistant、Function Call Output 等角色
- **tools**: 工具定义列表
- Response API 特有的特性: 可以挂载之前的 Response ID 进行上下文链接

4.2 Codex 的五类内置 Tools

通过分析 Codex 的 Tools Schema, 可以发现至少五种类型的内置 Function:

Codex 内置 Tools 分类

1. Shell 命令执行:

- **execute_command**: 执行 Shell 命令
- **send_input**: 向已有命令会话写入标准输入并读取标准输出

2. 计划管理:

- **update_plan**: 更新执行计划 (所有现代 Coding Agent 都内置 Plan Mode)

3. 用户交互:

- **request_user_input / ask_user_question**: 向用户询问问题

4. 代码操作:

- **apply_patch**: 编辑和执行代码
- **view_image**: 查看图片

5. 网络与搜索:

- **websearch**: 网络搜索

4.3 Multi-Agent (子代理) 工具

Codex 还内置了一套完整的 Multi-Agent 系统工具:

子代理管理工具

- **spawn_agent**: 启动一个新的子代理
- **send_input**: 给已有子代理继续发送消息或补充上下文
- **resume_agent**: 恢复上一个已关闭或暂停的子代理
- **wait**: 等待一个或多个子代理完成
- **close_agent**: 关闭子代理并返回其最后状态
- **spawn_agents_on_csv**: 按 CSV 每行批量启动子代理并汇总结果

CSV 批量子代理

`spawn_agents_on_csv` 是一个较新的功能，可以根据 CSV 文件的每一行批量启动子代理，每个子代理处理一行数据，最终汇总所有结果。这适用于批量处理类任务，是 Multi-Agent 编排的一种实用模式。

4.4 MCP 作为 Dynamic Tools

除内置 Tools 外，MCP (Model Context Protocol) 提供了动态工具机制：

- MCP Tools 取决于用户配置了哪些 MCP Server
- 每个 MCP Server 会暴露若干 Tools / Functions
- MCP 的 Tools 定义同样通过 API 的 `tools` 字段暴露给模型
- MCP 在配置文件中配置，但其暴露的 Tools Schema 不会出现在 Session 文件中

4.5 本章小结

Codex 内置五类 Tools (Shell 执行、计划管理、用户交互、代码操作、网络搜索)，以及完整的 Multi-Agent 子代理管理工具。MCP 提供动态工具扩展能力。所有 Tools 通过 API 的 `tools` 字段暴露给模型。

5 总结与延伸

5.1 Context Engineering 的全景图

本期完成了 Codex Context Engineering 的全景分析：

- **Skills**：出现在 User Message 的 content 中，采用渐进式加载
- **Tools**：出现在 API 的 `tools` 字段中，包括内置 Tools 和 MCP 动态 Tools
- **System 指令**：独立的 instructions 字段
- **消息历史**：多轮交互记录，可通过 compact 压缩
- **Memory**：持久化的记忆系统

三个关键认知

1. 不管是 Skills、MCP 还是预设 Tools，如果不暴露在 Context 中，模型就无法感知和调用
2. Skills 和 Tools 在技术上正交独立——Skills 在消息内容中，Tools 在工具字段中
3. 所有 Modern Agent 的核心工作都是 Context Engineering——精心管理送给模型的上下文

5.2 拓展阅读

- OpenAI Codex 官方文档中的 Agent Loop 设计
- Claude Code 的 CLAUDE.md 和 Skills 机制
- MCP (Model Context Protocol) 规范
- OpenAI Response API 文档